

Kaizen API – Configuração de Variáveis de Ambiente

Este projeto utiliza **variáveis de ambiente** para gerenciar chaves sensíveis (OpenAI, Pinecone, etc.), garantindo segurança e compatibilidade com o deploy no Render.

❌ O que não deve ser feito

- **Nunca** versionar chaves de API no código
- **Nunca** commitar arquivos `.env` com valores reais
- Não colocar tokens diretamente em scripts Python

O GitHub possui **Push Protection**, que bloqueia automaticamente commits contendo segredos.

✅ Estratégia correta (usada neste projeto)

1. Arquivo `.env` (apenas local)

O arquivo `.env` existe **somente para desenvolvimento local** e **não deve ser commitado**.

Exemplo de `.env`:

```
OPENAI_API_KEY=sk-xxxxxxx  
PINECONE_API_KEY=xxxxxxx  
PINECONE_ENVIRONMENT=us-east-1
```

⚠️ O `.env` deve estar listado no `.gitignore`

2. `.gitignore`

Confirme que o `.env` está ignorado:

```
.env
```

3. Uso das variáveis no código

O código **nunca** deve conter chaves diretamente. Ele apenas lê do ambiente:

```
import os

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
```

O mesmo código funciona **localmente** e **no Render**.

4. Configuração no Render (produção)

No Render:

1. Acesse o serviço
2. Vá em **Environment**
3. Cadastre as variáveis:
4. OPENAI_API_KEY
5. PINECONE_API_KEY
6. PINECONE_ENVIRONMENT

O Render injeta automaticamente essas variáveis no ambiente da aplicação.

Indexação de produtos

Sempre que for necessário indexar novos produtos:

- **Não é preciso alterar chaves**
- Basta rodar o script normalmente

```
python index_products.py
```

As chaves serão lidas do ambiente (local ou Render).

Resumo

- `.env` → apenas local
- Git → nunca recebe segredos
- Render → fonte oficial das chaves em produção
- Código → usa `os.getenv()`

Essa abordagem evita bloqueios do GitHub e mantém o projeto seguro.

✅ Setup validado e funcionando